

Ekspertski sistem baziran na pravilima za analizu stanja i dostižnosti ciljeva u igri Terraforming Mars

David Makan, SBNZ projekat

Motivacija

Terraforming Mars je kompleksna strateška društvena igra u kojoj igrači upravljaju korporacijama i zajednički teraformišu Mars podizanjem temperature, nivoa kiseonika i postavljanjem okeana. Pravila igre definišu precizne uslove koji moraju biti ispunjeni da bi se ostvario određeni ishod: osvajanje milestone-a, pobjeda u nagradi, legalan odigrani potez ili završetak igre. Upravo ova precizna, nedvosmislena priroda pravila igre čini Terraforming Mars pogodnim domenom za sistem baziran na znanju.

Pregled problema

Terraforming Mars definiše skup striktnih pravila koja regulišu stanje igre:

- Pet milestone-ova sa egzaktnim uslovima osvajanja definisanim u pravilniku
- Pet nagrada sa egzaktnim kategorijama poređenja definisanim u pravilniku
- 137 projektne karte sa precizno definisanim tagovima, preduslovima za igranje i efektima
- Tri globalna parametra (temperatura, kiseonik, okeani) sa egzaktnim finalnim vrednostima
- Precizna pravila o tome kada i kako igrač može da tvrdi milestone ili finansira nagradu

Problem koji sistem rešava je sledeći: dato kompleksno stanje igre sa više igrača, više karata, više resursa i više aktivnih događaja. Koji milestone-ovi su dostižni, koji igrač trenutno vodi u kojoj nagradi, da li je neka karta legalna za igranje, i koji uslovi za kraj igre su ispunjeni? Ova pitanja imaju jednoznačne odgovore koji proizilaze direktno iz pravilnika, ali je ručno praćenje svih ovih uslova istovremeno kognitivno zahtevno.

Metodologija rada

Ulaz u sistem

Postoje dve vrste ulaza. Prvu vrstu čine podaci o stanju igre koje korisnik unosi ručno pre svake generacije kroz React korisnički interfejs. Drugu vrstu čini stream događaja koji korisnik unosi kroz CEP sekciju interfejsa za svaku akciju igrača i protivnika.

Od podataka o globalnom stanju igre očekuju se: trenutna generacija (1 do 14), nivo kiseonika (0% do 14%), temperatura (od -30C do +8C), broj postavljenih okeana (0 do 9) i generacija u kojoj je temperatura poslednji put podignuta.

Od podataka o stanju svakog igrača očekuju se: Terraforming Rating, trenutni resursi (megakrediti, čelik, titanijum, biljke, energija, toplota), produkcija svakog resursa, odigrane karte sa tagovima, karte u ruci i postavljene pločice na tabli. Karte se unose po imenu, a sistem ih razrešava u pune objekte sa svim efektima putem baze podataka karata.

Od podataka o milestone-ovima i nagradama očekuju se: koji milestone-ovi su slobodni i ko je koje osvojio, koje nagrade su finansirane i po kojoj ceni. Sistem ograničava unos na maksimalno 3 osvojena milestone-a i 3 finansirane nagrade, u skladu sa pravilnikom.

Stream događaja sadrži tri tipa: CardPlayedEvent (koji igrač, koji tag, timestamp generacije), TilePlayedEvent (koji igrač, tip pločice, timestamp generacije) i TemperatureRaisedEvent (timestamp generacije u kojoj je temperatura poslednji put podignuta).

Izlaz iz sistema

Sistem generiše pet kategorija izlaza prikazanih u korisničkom interfejsu rangirane po prioritetu URGENT, HIGH ili MEDIUM.

Milestone upozorenja obaveštavaju igrača o milestone-ovima koji su blizu dostizanja (sopstveni ili protivnički) zasnovano na egzaktnim pravilima pravilnika. **Analiza nagrada** prikazuje procenu vodstva u svakoj od pet nagradnih kategorija na osnovu trenutnog stanja igre. **Sinergije karata** detektuju karte u ruci čiji preduslovi za bonus efekat su ispunjeni prema tekstu karte, generisane iz baze podataka karata. **Pretnje od protivnika** su upozorenja o detektovanim obrascima ponašanja koji ukazuju na blizinu osvajanja milestone-a ili dominacije u nagradi. **Projekcija VP** prikazuje procenu ukupnog rezultata igrača na kraju igre iz svih izvora. Za svaku preporuku prikazuje se obrazloženje koje navodi koji lanac pravila se aktivirao, koji pravilnik uslov je ispunjen i kojim redom su zaključci izvedeni.

Baza znanja — Pravila

Legenda

GameState — globalno stanje igre: generacija, kiseonik, temperatura, broj okeana, poslednja generacija podizanja temperature.

PlayerState — stanje igrača: TR, svi resursi, sve produkcije, broj gradova, zelenila, science i building tagova, ID igrača.

PlayedCard — odigrana karta sa svim tagovima, VP vrednošću, ID igrača koji ju je odigrao i generacijom igranja.

CardInHand — karta u ruci igrača sa preduslovima, tagovima i efektima: placesCity, placesGreenery, raisesTemperature, requiresEnergy, scienceSynergyBonus, povećanja produkcija, cena.

TilePlaced — činjenica da je pločica tipa CITY, GREENERY ili SPECIAL postavljena.

Milestone — stanje jednog milestonea: tip, egzaktni pravilnik uslov, ko ga je osvojio.

Award — stanje nagrade: tip, egzaktni pravilnik uslov za pobednika, ko je finansirao.

VP – victory point: bodovi na kraju igre.

CardPlayedEvent — CEP event odigravanja karte (playerId, tag, timestamp generacije).

TilePlayedEvent — CEP event postavljanja pločice (playerId, tileType, timestamp).

TemperatureRaisedEvent — CEP event podizanja temperature (timestamp generacije).

TagCount — agregirani broj tagova određenog tipa za jednog igrača.

Insight — međuzaključak prvog ili drugog nivoa umetnut pravilom u radnu memoriju.

Alert — upozorenje višeg nivoa nastalo kombinovanjem više Insight-ova.

ThreatAlert — upozorenje generisano CEP pravilima o obrascu ponašanja protivnika.

Recommendation — finalna preporuka prikazana korisniku sa prioritetom i obrazloženjem.

StrategicAdvice — visokonivoni savet generisan kombinovanjem više Recommendation-a.

MilestoneReport — izveštaj o dostižnosti milestonea generisan backward chaining query-jem.

ScoreProjection – projekcija ukupnih VP iz svih izvora za jednog igrača.

UsedCards – pomoćni objekat koji prati koje karte su već iskorišćene u jednoj grani backward chaining rekurzije, sprečava dvostruko brojanje iste karte.

Pravila

Forward Chaining (4 nivoa zaključivanja)

Sistem sadrži 22 pravila koja se automatski aktiviraju u kaskadi. Sistem koristi četvorostepeni lanac. Nivo 1 čita direktno stanje igre i umeće Insight i TagCount činjenice. Nivo 2 kombinuje više Insight-ova i umeće Alert činjenice. Nivo 3 čita Alert činjenice i generiše Recommendation činjenice vidljive korisniku. Nivo 4 sintetizuje više Recommendation-a u StrategicAdvice činjenice.

Nivo 1: Detekcija stanja (10 pravila)

Sva pravila ovog nivoa čitaju direktno stanje igre i umetaju Insight ili TagCount činjenice. Nijedan od ovih pravila ne zavisi od drugog pravila. Aktiviraju se direktno na osnovu unetih podataka.

FC-1 (Naučni engine):

Detektuje se ako igrač ima 3 ili više naučna taga kao preduslov za bonus efekat. Prag od 3 dolazi iz najnižeg navedenog preduslova među tim kartama. Sistem koristi accumulate funkciju unutar when bloka da prebroji PlayedCard objekte sa SCIENCE tagom.

- kada: accumulate(PlayedCard(playerId == trenutniIgrač, tags sadrži SCIENCE), count) >= 3
- tada:
 - o umetni Insight(SCIENCE_ENGINE_ACTIVE) – ovo Insight će u nivou 2 omogućiti detekciju blokiranih karata

FC-2 (Energetsko usko grlo):

Pravilnik str. 9: energija se ne prenosi između generacija. Produkcija ispod 2 znači da igrač ne može napajati ni dve energy-cost karte u istoj generaciji.

- kada: PlayerState.energyProduction < 2 za trenutnog igrača
- tada:
 - o umetni Insight(ENERGY_BOTTLENECK) – ovo Insight će u nivou 2 učestvovati u detekciji blokiranih karata

FC-3 (Nestašica novca):

Minimalna cena karata u standardnoj igri je 6 MC, a prosečna cena karata u projektu iznosi 13 MC izračunato na osnovu baze karata. Prag od 10 MC znači da igrač ne može priuštiti ni prosečnu kartu ove generacije.

- kada: PlayerState.megacredits < 10 za trenutnog igrača
- tada:
 - o umetni Insight(MC_SHORTAGE)

FC-4 (Mayor pretnja):

Pravilnik str. 5: Mayor zahteva tačno 3 grada. Sa 2 grada protivnik je jednu akciju od ispunjavanja uslova i osvajanja milestone-a. Sistem koristi accumulate za brojanje TilePlaced(CITY) za protivnika.

- kada:
 - o accumulate(TilePlaced(playerId == protivnik, CITY), count) >= 2 i Milestone(MAYOR).claimed == false
- tada:

- umetni Insight(OPPONENT_NEAR_MAYOR)

FC-5 (Pretnja od protivnika):

Pravilnik str. 5: Gardener zahteva tačno 3 zelenila. Sa 2 zelenila protivnik je jednu akciju od osvajanja.

- kada:
 - accumulate(TilePlaced(playerId == protivnik, GREENERY), count) >= 2 i Milestone(GARDENER).claimed == false
- tada:
 - umetni Insight(OPPONENT_NEAR_GARDENER)

FC-6 (Builder pretnja):

Pravilnik str. 5: Builder zahteva tačno 8 building tagova. Sa 6 tagova protivnik je 1 do 2 karte od ispunjavanja uslova, dovoljno blizu da se reaguje. Sistem koristi accumulate za brojanje PlayedCard sa BUILDING tagom za protivnika, bez oslanjanja na TagCount činjenice koje generišu kasnija pravila.

- kada:
 - accumulate(PlayedCard(playerId == protivnik, tags sadrži BUILDING), count) >= 6 i Milestone(BUILDER).claimed == false
- tada:
 - umetni Insight(OPPONENT_NEAR_BUILDER)

FC-7a/7b/7c/7d/7e (Sopstvena blizina cilju – 5 zasebnih pravila):

Pravilnik str. 5: egzaktni pragovi svih 5 milestone-ova. Pravilo se aktivira kada igrač dostigne 80% svakog uslova, što znaci 1 do 2 akcije od cilja. Zbog toga što svaki milestone zahteva različit tip Insight-a i različite podatke iz radne memorije, implementirano je kao 5 zasebnih pravila umesto jednog pravila sa OR granama.

- FC-7a: PlayerState.terraformRating >= 28 → Insight(SELF_NEAR_TERRAFORMER)
- FC-7b: PlayerState.cityCount >= 2 → Insight(SELF_NEAR_MAYOR)
- FC-7c: PlayerState.greeneryCount >= 2 → Insight(SELF_NEAR_GARDENER)
- FC-7d: PlayerState.buildingTagCount >= 6 → Insight(SELF_NEAR_BUILDER)
- FC-7e: accumulate(CardInHand(playerId == igrač), count) >= 13 → Insight(SELF_NEAR_PLANNER)

FC-8 (Stagnacija temperature):

Pravilnik str. 4 do 5: temperatura mora dostići +8°C da bi igra završila. Stagnacija 2 ili više generacija direktno produljuje trajanje igre i prednost daje igračima koji kontrolišu tempo podizanja parametara.

- kada:
 - `(GameState.generation - GameState.lastRaisedGeneration) >= 2`
- tada:
 - `umetni Insight(TEMPERATURE_STALLED)`

FC-9 (Prebrojavanje SCIENCE tagova po igraču):

Pravilnik str. 6: Scientist nagrada ide igraču sa najviše naučnih tagova na kraju igre. TagCount činjenica je neophodna za poređenje između igrača u pravilima nivoa 2. Sistem koristi accumulate funkciju koja prolazi kroz sve odigrane karte jednog igrača, filtrira one sa naučnim tagom i vraća ukupan broj u jednom koraku. Pravilo se aktivira za sve igrače (i trenutnog i protivnike).

- kada:
 - `accumulate(PlayedCard(playerId == bilo_koji_igrač, SCIENCE), count)`
- tada:
 - `TagCount(playerId, SCIENCE, count)`

FC-10 (Prebrojavanje BUILDING tagova po igraču):

Pravilnik str. 5: Builder milestone zahteva 8 building tagova. TagCount činjenica je neophodna za praćenje napretka. Aktivira se za sve igrače.

- kada:
 - `accumulate(PlayedCard(playerId == bilo_koji_igrač, BUILDING), count)`
- tada:
 - `TagCount(playerId, BUILDING, count)`

Nivo 2: Kombinovanje u upozorenja (5 pravila)

Pravila ovog nivoa aktiviraju se isključivo na osnovu Insight i TagCount činjenica umetnutih od strane pravila nivoa 1. Ne čitaju direktno stanje igre.

FC-11 (Blokirana karta):

Karta Fusion Power eksplicitno zahteva i energijsku produkciju i science synergyj kao preduslove. Oba uslova moraju biti ispunjena da bi karta bila igriva.

- kada:
 - Insight(SCIENCE_ENGINE_ACTIVE) i Insight(ENERGY_BOTTLENECK) i CardInHand(requiresEnergy == true, scienceSynergyBonus == true)
- tada:
 - umetni Alert(CARD_BLOCKED, ta_karta)

FC-12 (Pretnja za Mayor milestone):

Pravilnik str. 5: ako protivnik ima 2 grada a igrač ima manje od 2, protivnik će u narednoj akciji moći da osvoji milestone.

- kada:
 - Insight(OPPONENT_NEAR_MAYOR) i PlayerState.cityCount < 2
- tada:
 - umetni Alert(MAYOR_THREAT, URGENT)

FC-13 (Pretnja za Gardener milestone):

Ista logika kao FC-12 primenjena na Gardener milestone.

- kada:
 - Insight(OPPONENT_NEAR_GARDENER) i PlayerState.greeneryCount < 2
- tada:
 - umetni Alert(GARDENER_THREAT, URGENT)

FC-14a/14b (Procena pozicije u Scientist nagradi – 2 pravila):

Pravilnik str. 6: Scientist nagrada ide igraču sa najviše naučnih tagova (5 VP), a drugi igrač dobija 2 VP.

- kada:
 - TagCount(igrač, SCIENCE) > TagCount(svi protivnici, SCIENCE)
- tada:
 - umetni Alert(SCIENTIST_AWARD_FAVORABLE, HIGH)

- kada:
 - TagCount(provnik, SCIENCE) > TagCount(igrač, SCIENCE) i razlika <= 2
- tada:
 - umetni Alert(SCIENTIST_AWARD_CONTESTED, MEDIUM)

FC-15 (Kritična stagnacija teraformisanja):

Pravilnik str. 4: temperatura stagnira i kiseonik je ispod 8% posle generacije 7 znači da dva od tri parametra kasne.

- kada:
 - Insight(TEMPERATURE_STALLED) i GameState.oxygenLevel < 8 i GameState.generation > 7
- tada:
 - umetni Alert(TERRAFORMING_PACE_CRITICAL, URGENT)

Nivo 3: Konkretno preporuke (5 pravila)

Pravila ovog nivoa čitaju Alert činjenice i generišu Recommendation činjenice koje se prikazuju korisniku sa prioritetom i tekstualnim obrazloženjem.

FC-16 (Preporuka za energetska infrastrukturu):

Alert CARD_BLOCKED znači da vredna karta nije igriva zbog nedostatka energije, direktna posledica preduslova navedenih na karti. Rešenje je igranje energy-producing karte iz ruke.

- kada:
 - Alert(CARD_BLOCKED) i CardInHand(energyProductionIncrease > 0)
- tada:
 - umetni Recommendation(PLAY_CARD, energetska_karta, prioritet HIGH)

FC-17 (Preporuka za kontestovanje Mayor milestone-a):

Pravilnik definiše da milestone donosi 5 VP osvajačiju. Postoji samo jedan Mayor milestone po partiji, ako ga protivnik osvoji, taj bonus više nije dostupan ni jednom drugom igraču.

- kada:
 - Alert(MAYOR_THREAT) i CardInHand(placesCity == true)
- tada:
 - umetni Recommendation(PLAY_CARD, karta_za_grad, prioritet URGENT)

FC-18 (Preporuka za finansiranje Scientist nagrade):

Pravilnik str. 6 definiše da prvi igrač plaća 8 MC za finansiranje nagrade i da pobednik nagrade osvaja 5 VP. Ako igrač vodi u science tagovima i nagrada nije finansirana, ispunjeni su uslovi za izvršavanje akcije finansiranja nagrade definisane pravilnikom.

- kada:
 - Alert(SCIENTIST_AWARD_FAVORABLE) i Award(SCIENTIST).funded == false i PlayerState.megacredits >= 8

- tada:
 - umetni Recommendation(FUND_AWARD, SCIENTIST, prioritet HIGH)

FC-19 (Preporuka za izbegavanje kontestovane nagrade):

Pravilnik str. 6 definiše da drugi igrač u nagradi dobija samo 2 VP, a finansiranje nagrade košta 14 MC za drugog finansijera. Ako igrač zaostaje za više od 3 taga, dostizanje prvog mesta zahteva više akcija nego što je cena finansiranja opravdava.

- kada:
 - Alert(SCIENTIST_AWARD_CONTESTED) i razlika tagova > 3
- tada:
 - umetni Recommendation(AVOID_AWARD, SCIENTIST, prioritet MEDIUM)

FC-20a (Preporuka za hitno teraformisanje – konverzija toplote):

Pravilnik str. 8 definiše da se 8 toplote može konvertovati u povišenje temperature za 1 stepen kao standardna akcija dostupna svakom igraču. Kada je detektovano stanje TERRAFORMING_PACE_CRITICAL, ova akcija ima prednost nad razvojem engine-a.

- kada:
 - Alert(TERRAFORMING_PACE_CRITICAL) i PlayerState.heat >= 8
- tada:
 - umetni Recommendation(RAISE_TEMPERATURE, URGENT, „konvertuj toplotu“)

FC-20b (Preporuka za hitno teraformisanje – karta):

Isto kao prethodno pravilo, samo umesto konvertovanja toplote, odigraj kartu koja može podići temperaturu.

- kada:
 - Alert(TERRAFORMING_PACE_CRITICAL) i PlayerState.heat < 8 i CardInHand(raisesTemperature == true)
- tada:
 - umetni Recommendation(RAISE_TEMPERATURE, URGENT, karta)

Nivo 4: Strateška sinteza (2 pravila)

Pravila ovog nivoa aktiviraju se kada više Recommendation činjenica zajedno ukazuju na istu stratešku situaciju. Generišu StrategicAdvice činjenice koje su zaključci višeg reda apstrakcije.

FC-21 (Preporuka za pivot na energetski engine):

Ako sistem generiše 3 ili više HIGH preporuka za POWER karte (detektovano accumulate-om nad Recommendation činjenicama), sintetički zaključak je da se ceo fokus generacije treba usmeriti na energiju.

- kada:
 - o accumulate(Recommendation(PLAY_CARD, HIGH) gde karta ima POWER tag, count) >= 3
- tada:
 - o umetni StrategicAdvice(ENERGY_ENGINE_PIVOT)

FC-22 (Preporuka za defanzivnu generaciju):

Detektuje situaciju u kojoj je istovremeno aktivno više URGENT upozorenja.

- kada:
 - o accumulate(Alert(priority == URGENT), count) >= 2
- tada:
 - o umetni StrategicAdvice(DEFENSIVE_GENERATION)

Accumulate

Accumulate funkcija se koristi za agregaciju grupe činjenica iz radne memorije u jedan zaključak. Za razliku od forward chaining pravila koja reaguju na pojedinačne činjenice, accumulate prolazi kroz skup činjenica, primenjuje agregatnu funkciju i vraća jedan rezultat koji se dalje koristi u pravilu. Sistem koristi accumulate na više mesta u forward chaining pravilima, ali u jednom slučaju je istaknuto kao ključna agregacija.

ACC-1: Projekcija ukupnih VP iz svih izvora

Pravilnik str. 7 do 8: konačni rezultat igrača je zbir Terraforming Ratinga, VP poena sa odigranih karata, VP od pločica (zelenila donose 1 VP svako, gradovi donose 1 VP kao aproksimacija), VP od osvojenih milestone-ova (5 VP svaki) i VP od nagrada (5 VP za pobednika — optimistična procena).

Sistem prolazi kroz sve izvore VP u radnoj memoriji koristeći pet zasebnih accumulate poziva:

- sum(PlayedCard.vpValue) za kartne VP
- count(TilePlaced(GREENERY)) za VP zelenila
- count(TilePlaced(CITY)) za VP gradova
- count(Milestone(claimedBy == igrač)) za milestone VP
- count(Award(fundedBy == igrač)) za award VP

Pravilo se aktivira za svakog igrača (ne samo trenutnog), jer je poređenje projekcija između igrača neophodno za pravilo ACC-1b. Kada je projekcija izračunata, umeće se ScoreProjection činjenica u radnu memoriju.

ACC-1b (Downstream pravilo – zaostajanje u VP)

Ako igrač zaostaje za protivnikom za više od 10 VP u projekciji, aktivira se Insight(SCORE_LAGGING) koji upozorava igrača na potrebu agresivnije strategije.

CEP — Obrada kompleksnih događaja (3 pravila)

CEP pravila prate stream događaja kroz vreme i detektuju obrasce koji nisu vidljivi posmatranjem trenutnog stanja. Ključna razlika u odnosu na forward chaining je vremenska dimenzija: sistem ne pita šta je stanje sada već šta se dešavalo tokom poslednjih N generacija. Svi događaji imaju timestamp koji odgovara broju generacije u kojoj su nastali, konvertovan u milisekunde (1 generacija = 1000ms). KieSession koristi pseudo sat koji se pomera na trenutnu generaciju pre evaluacije pravila.

CEP-1: Detekcija science rush-a protivnika (sliding window)

Igrač koji igra 3 science karte u 2 generacije aktivno gradi science engine. Statičko pravilo vidi samo ukupan broj tagova bez vremenske dimenzije.

- događaj: CardPlayedEvent(playerId == protivnik, tag == SCIENCE)
- kada: sliding window od 2000ms (2 generacije), count >= 3
- tada: umetni ThreatAlert(targetPlayerId, sourcePlayerId, SCIENCE_RUSH)

CEP-2: Detekcija Mayor rush-a protivnika (sekvencija događaja)

Dva grada postavljena u dve bliske generacije je namerni Mayor rush obrazac koji statičko pravilo ne može detektovati jer vidi samo ukupan broj gradova.

- događaj: TilePlayedEvent(playerId == protivnik, tileType == CITY)
- kada: postoje dva TilePlayedEvent za protivnika gde je drugi u okviru 2 generacije od prvog (generation - g1 <= 2) i oba su u okviru poslednjih 2 generacije od trenutne
- tada: umetni ThreatAlert(targetPlayerId, sourcePlayerId, MAYOR_RUSH)

CEP-3a/3b: Detekcija izgubljenog temperaturnog tempa (odsustvo događaja – 2 pravila)

Odsustvo događaja u vremenskom intervalu je fundamentalno nemoguće detektovati statičkim pravilom. Implementirano kroz dva zasebna pravila koja pokrivaju dve različite situacije:

- **CEP-3a:** nema TempRaisedEvent u poslednjih 2 generacije i igrač ima heat >= 8 → Insight(TEMPERATURE_STALLED, "toplota dostupna za konverziju")

- **CEP-3b:** nema TempRaisedEvent u poslednjih 2 generacije i postoji
 CardInHand(raisesTemperature == true, cost <= megacredits) →
 Insight(TEMPERATURE_STALLED, "karte dostupne")

Backward Chaining (5 query-ja)

Backward chaining se koristi za goal-directed rezonovanje. Sistem postavlja cilj i zaključuje unazad da li je taj cilj dostižan i koji poduslovi moraju biti ispunjeni. U Terraforming Mars-u ovo odgovara egzaktnim pitanjima o milestone-ovima i nagradama definisanim pravilnikom.

Za razliku od forward chaining-a koji kreće od činjenica ka zaključku, backward chaining kreće od cilja i pokušava da dokaže potrebne podciljeve. Rekurzivnost se ostvaruje pozivanjem istog query-ja sa ažuriranim parametrima.

Svaki backward chaining query koristi objekat UsedCards koji prati koje karte su već iskorišćene u jednoj rekurzivnoj grani, sprečavajući dvostruko računanje iste fizičke karte. Parametar remainingMc se smanjuje za cenu svake karte pri svakom rekurzivnom koraku, obezbeđujući ispravno modelovanje budžeta. Tamo gde standard projekat troši biljke, parametar remainingPlants se smanjuje za 8 pri svakom koraku.

Zbog poznatog ograničenja Drools 7 gde multiple or grane u query-ju koje sve rekurzivno pozivaju isti query mogu ometati jedna drugu kada jedna grana ne pronađe odgovarajuće činjenice, system koristi arhitekturu podeljenih query-ja: svaka rekurzivna putanja ima sopstveni podquery, a standardni projekat poziva poseban JustStandard query bez grane za karte.

Forward pravilo koje koristi BC query takođe proverava da ukupan broj već osvajenih milestone-ova ili finansiranih nagrada nije dostigao maksimum od 3 (pravilnik: maksimalno 3 milestone-a i 3 nagrada mogu biti osvojena po partiji).

BC-1: canClaimMilestone — Mayor (3 grada)

canClaimMayor(playerId, currentCities, 3, remainingMc, usedCards)

└─ canClaimBaseCase: currentCities >= 3 → TRUE

└─ canClaimMayorViaCard:

| └─ CardInHand(placesCity == true, cost <= remainingMc, id not in usedCards)

| └─ canClaimMayor(playerId, currentCities+1, 3, remainingMc-cost, usedCards+id)

└─ canClaimMayorViaStandardProject:

└─ remainingMc >= 25

└─ canClaimMayorJustStandard(playerId, currentCities+1, 3, remainingMc-25, usedCards)

└─ canClaimBaseCase: currentCities >= 3 → TRUE

└─ canClaimMayorViaStandardProject (rekurzija samo kroz standardni projekat)

BC-2: canClaimMilestone — Builder (8 building tagova)

Builder nema standardni projekat, jedini način je kroz karte sa BUILDING tagom.

canClaimBuilder(playerId, currentTags, 8, remainingMc, usedCards)

- └— canClaimBaseCase: currentTags >= 8 → TRUE
- └ canClaimBuilderViaCard:
 - └— CardInHand(tags contains BUILDING, cost <= remainingMc, id not in usedCards)
 - └ canClaimBuilder(playerId, currentTags+1, 8, remainingMc-cost, usedCards+id)

BC-3: canClaimMilestone — Gardener (3 zelenila)

Standardni projekat GREENERY košta 8 biljaka. Parametar remainingPlants se smanjuje za 8 pri svakom koraku standardnog projekta, što sprečava beskonačnu rekurziju i korektno modeluje trošak biljaka.

canClaimGardener(playerId, currentGreens, 3, remainingMc, remainingPlants, usedCards)

- └— canClaimBaseCase: currentGreens >= 3 → TRUE
- └— canClaimGardenerViaCard:
 - | └— CardInHand(placesGreenery == true, cost <= remainingMc, id not in usedCards)
 - | └ canClaimGardener(playerId, currentGreens+1, 3, remainingMc-cost, remainingPlants, usedCards+id)
- └ canClaimGardenerViaStandardProject:
 - └— remainingPlants >= 8
 - └ canClaimGardenerJustStandard(playerId, currentGreens+1, 3, remainingMc, remainingPlants-8, usedCards)
 - └— canClaimBaseCase: currentGreens >= 3 → TRUE
 - └ canClaimGardenerViaStandardProject (rekurzija samo kroz standardni projekat)

BC-4: canWinAward — Scientist (najviše SCIENCE tagova)

Sistem direktno broji science tagove iz PlayedCard objekata (accumulate unutar when bloka), a maksimalan broj tagova protivnika se računa kroz QueryUtils.getMaxOpponentScienceTags() koji prolazi kroz sve odigrane karte. Forward pravilo takođe proverava da je cena finansiranja nagrade ispunjena (8/14/20 MC zavisno od broja već finansiranih nagrada).

canWinScientist(playerId, myCount, opponentMax, remainingMc, usedCards)

└— canWinBase: myCount > opponentMax → TRUE

└— canWinScientistViaCard:

└— CardInHand(tags contains SCIENCE, cost <= remainingMc, id not in usedCards)

└— canWinScientist(playerId, myCount+1, opponentMax, remainingMc-cost, usedCards+id)

BC-5: canWinAward — Banker (najviša MC produkcija)

Maksimalna MC produkcija protivnika se dobija accumulate(max) nad svim PlayerState objektima koji nisu trenutni igrač. Karte koje povećavaju MC produkciju imaju polje mcProductionIncrease > 0.

canWinBanker(playerId, myProduction, opponentMax, remainingMc, usedCards)

└— canWinBase: myProduction > opponentMax → TRUE

└— canWinBankerViaCard:

└— CardInHand(mcProductionIncrease > 0, cost <= remainingMc, id not in usedCards)

└— canWinBanker(playerId, myProduction+mcProductionIncrease, opponentMax, remainingMc-cost, usedCards+id)

Template pravila

Template mehanizam rešava problem skalabilnosti sinergija karata. Terraforming Mars ima više od 200 karata (u osnovnoj igri 137) od kojih mnoge imaju bonus efekte koji se aktiviraju kada igrač ima određeni broj tagova određenog tipa. Ručno pisanje jednog pravila po karti nije održivo.

Implementacija koristi Java StringBuilder za direktno generisanje DRL teksta iz baze podataka, umesto klasičnog .drt fajla i ObjectDataCompiler koji je pokazao nestabilnost u Drools 7.49.0.Final pri programatskom korišćenju sa List<Map> parametrima. Generisanje se aktivira putem Spring @EventListener(ApplicationReadyEvent.class) mehanizma, koji garantuje da se baza podataka (uključujući card_synergy tabelu) popuni pre nego što se template pravila generišu i kompajliraju.

Baza podataka sadrži tabelu card_synergy sa kolonama: card_name, required_tag, threshold, priority, effect_text. Svaki red generiše jedno pravilo oblika:

kada:

- karta [card_name] se nalazi u ruci igrača i broj PlayedCard sa tagom [required_tag] za tog igrača >= [threshold]

tada:

- umetni Recommendation(ACTION_ENABLED_BY_CARD_CONDITION, [card_name],
prioritet [priority], '[effect_text] (trenutno imaš X [required_tag] tagova)')

Primeri iz baze podataka koji generišu pravila:

- Fusion Power (tag: SCIENCE, prag: 2, prioritet: HIGH) — tekst karte: requires energy and 2 science tags
- Ecological Zone (tag: PLANT, prag: 1, prioritet: MEDIUM) — tekst karte: bonus with plant or animal tags
- Ecological Zone (tag: ANIMAL, prag: 1, prioritet: MEDIUM) — drugi red za isti kartu, drugi tag
- Decomposers (tag: MICROBE, prag: 3, prioritet: HIGH) — tekst karte: microbe resource bonus activates with 3+ microbe tags
- Kelp Farming (tag: OCEAN, prag: 6, prioritet: HIGH) — tekst karte: production bonus requires 6+ oceans

Klasni diagram

